# Plain-Language Glossary

This appendix explains what the report's common machine-learning and software terms mean, why they appear in CodeMentor-LLM, and what the measurements can and cannot tell you.

## A. Training and model terms

| Term | Meaning and why it is used here |
|---|---|
| **LLM (Large Language Model)** | A neural network trained on large amounts of text to predict and generate the next token. Here, Llama 3.2 3B Instruct is the starting model, adapted for coding help. |
| **Fine-tuning** | Continuing training of an already-trained model on a narrower dataset or task. It adapts existing language ability instead of training a model from zero. |
| **SFT (Supervised Fine-Tuning)** | Training on examples that pair a prompt with a desired answer. The model learns to imitate those answers. This is the project's first adaptation stage, using coding instruction examples. |
| **Epoch** | One full pass over the training examples. Three SFT epochs means the selected SFT examples are revisited three times; one DPO epoch means one pass over preference examples. More epochs are not automatically better: they can overfit. |
| **Batch size and gradient accumulation** | Batch size is the number of examples processed in one device step. Gradient accumulation adds several step gradients before updating weights. With batch size 4 and accumulation 4, the effective batch is 16 per device. |
| **Learning rate and warmup** | Learning rate controls how large each training update is. Warmup gradually raises it at the beginning to make early training more stable. The SFT config uses 2e-4 with 5% warmup; DPO uses 5e-5. |
| **Optimizer and scheduler** | The optimizer applies weight updates; AdamW is an adaptive optimizer with weight decay. Paged AdamW helps manage GPU memory spikes. A cosine scheduler gradually lowers the learning rate following a cosine-shaped curve. |
| **FP16 and BF16** | 16-bit floating-point number formats that use less memory than 32-bit values. FP16 is configured for training; BF16 is the compute type for 4-bit model loading. They are precision formats, not model sizes. |
| **LoRA (Low-Rank Adaptation)** | A parameter-efficient fine-tuning method. It freezes the large pretrained weights and learns small low-rank matrices that modify selected layers. This cuts the number of trainable weights and saves memory; the project targets attention projections q/k/v/o. |
| **Rank r, alpha, dropout, adapter** | Rank r controls the size/capacity of the LoRA update (here r=16); alpha scales that update (32); dropout (0.05) randomly drops adapter activations during training to reduce overfitting. The adapter is the small learned set of LoRA weights. |
| **QLoRA** | Quantized Low-Rank Adaptation: the base model is loaded in 4-bit form and kept frozen while LoRA adapters are trained. This combines 4-bit memory savings with small trainable updates, making fine-tuning possible on a constrained GPU. |
| **Quantization** | Representing model weights with fewer bits. It reduces memory use and can make deployment fit a smaller GPU, with some possible loss or change in model behavior. The project uses 4-bit weights. |
| **NF4 (4-bit NormalFloat)** | A 4-bit, non-uniform number format designed for weights that are approximately normally distributed. Its levels are allocated to represent that distribution efficiently. The project selects NF4 to compress the frozen base weights for QLoRA, reducing GPU memory while retaining useful model quality. |
| **Double quantization** | Quantizes the small scaling constants used by blockwise 4-bit quantization, saving additional memory. It is enabled in this project's configuration. |
| **Causal language model and tokens** | A causal model predicts the next token using the tokens before it. A token is a word fragment or symbol, not necessarily a whole word. max_new_tokens caps how many new tokens can be generated. |
| **Sequence length** | The maximum token context processed for a training example. SFT allows 2,048 tokens; DPO allows 512 total tokens and up to 256 prompt tokens. Longer examples may be truncated or excluded depending on pipeline behavior. |

## B. Preference learning and evaluation

| Term | Meaning and why it is used here |
|---|---|
| **DPO (Direct Preference Optimization)** | A training method that uses pairs of answers labelled preferred (chosen) and less preferred (rejected). It adjusts the model to favor the chosen answer directly, without a separate reward-model training stage. Here it follows SFT and uses beta=0.1. |
| **Chosen and rejected responses; beta** | Chosen is the answer the training data says is better; rejected is the less-preferred answer. Beta controls how strongly DPO balances preference learning against staying near the reference model. The project config sets beta to 0.1. |
| **ROUGE** | Recall-Oriented Understudy for Gisting Evaluation. A family of text-overlap metrics originally used mainly to compare machine summaries with reference summaries. It is used here to compare generated coding answers with reference answers; a higher overlap score is not proof that code runs or is correct. |
| **ROUGE-1, ROUGE-2, ROUGE-L** | ROUGE-1 measures overlapping single words (unigrams); ROUGE-2 measures overlapping adjacent word pairs (bigrams); ROUGE-L uses the longest common subsequence, so it reflects shared word order. The project reports all three to compare answer similarity at different granularities. |
| **BERTScore, precision, recall, F1** | BERTScore compares contextual token representations from a pretrained language model rather than requiring exact word matches. Precision reflects how much generated content matches reference content; recall reflects how much reference content is covered; F1 combines both. It can recognize paraphrases, but cannot certify factual or code correctness. |
| **Test split and sample size** | The test split is held back from training for final evaluation. The repository's metric files report 50 test examples, a small sample that gives an initial comparison but may not represent all coding tasks. |
| **Loss and steps** | Loss is the training objective's error signal; lower is not by itself proof of better answers. A step is one optimizer update. The recorded DPO loss/12 steps are experiment notes, not a standalone quality guarantee. |

## C. Inference and application terms

| Term | Meaning and why it is used here |
|---|---|
| **Inference and decoding** | Inference is using the trained model to answer a new prompt. Decoding is how token predictions are selected. In this project, the Modal service uses greedy decoding (do_sample=false); the general config's sampling settings are not active there. |
| **Temperature, top-p, sampling** | When sampling is enabled, temperature adjusts randomness (lower is more predictable), and top-p keeps a probability-ranked set of likely next tokens until their cumulative probability reaches p. The general config lists temperature 0.7 and top_p 0.9, but Modal currently uses do_sample=false, so they do not control deployed output. |
| **Repetition penalty** | A decoding adjustment intended to discourage repeating text. The local inference/evaluation code sets 1.3; the active Modal generation code does not apply it. |
| **Merged model** | The final model artifact produced by combining trained adapter updates with the base model weights. The repository documents the merged model on Hugging Face Hub for serving. |
| **Latency** | Elapsed time for a response, usually reported in milliseconds. It can vary with prompt length, output length, GPU load, network time, and cold starts. |
| **API, REST, endpoint, JSON** | An API lets programs communicate. REST is a common web API style using HTTP methods such as GET and POST. An endpoint is one address/action, such as POST /generate. JSON is the structured request/response format used here. |
| **Server-Sent Events (SSE) and streaming** | SSE is a one-way HTTP stream from server to browser. Streaming sends generated text in chunks as it becomes available instead of waiting for the complete answer; the project's stream ends with a [DONE] marker. |
| **FastAPI and Pydantic** | FastAPI is the Python web framework that hosts the API. Pydantic defines and validates request/response data, such as prompt length and allowed token limits. |
| **SQLite and SQLAlchemy** | SQLite is a small database stored in a local file; SQLAlchemy is the Python ORM that maps code objects to database tables. This project uses it to log requests, responses, latency, time, and success status, not to save full conversations. |
| **Streamlit** | A Python framework used to build the project's web interface: question entry, streamed answer display, appearance controls, and request log. |
| **Modal, GPU, cold start** | Modal hosts the model in an on-demand cloud container. A GPU accelerates model calculations. A cold start is the delay when a new container starts and loads the model; warm requests reuse an already-running container. |
| **Hugging Face Hub and Weights & Biases** | Hugging Face Hub stores model and dataset artifacts. Weights & Biases (W&B) records experiment configuration and training metrics. |
| **Docker, CI/CD, CORS** | Docker packages an app and its dependencies into a container. CI/CD means Continuous Integration/Continuous Delivery: automated checks/builds on code changes. CORS (Cross-Origin Resource Sharing) is a browser rule that controls which website origins may call an API; the current API config allows all origins. |